

CIE-1 PYQ Bank: Model Answers

Handwritten Question Bank & Solutions

Syllabus Topics Covered:

Table of Contents

Unit 1 Examination Questions & Answers	p. 2
Unit 2 Examination Questions & Answers	p. 14
Out of CIE-1 Scope Questions (10 Questions)	p. 24

Unit 1 Examination Questions & Answers

UNIT 1

Verified model solutions for Unit 1 past exam questions.

Unit 1 4 Marks High Priority

SEE JAN 2026

Theory Link: Unit 1 § 1

Explain the role of ReactDOM in connecting React components with the browser DOM.

Practice Problem:

Model Exam Answer

Solution:

1. Role of ReactDOM

ReactDOM acts as the bridge or rendering engine between React's declarative component tree (Virtual DOM) and the actual browser Document Object Model (Real DOM).

- **Root Creation and Mounting:** In modern React 18+, `ReactDOM.createRoot(container)` initializes a React root on an existing HTML container node (usually `<div id="root"></div>`).
- **Rendering Virtual Trees:** `root.render(<App />)` takes the top-level React element, evaluates its component tree, and builds the in-memory Virtual DOM representation.
- **DOM Reconciliation & Commit:** When state changes, ReactDOM receives the reconciliation diff from React and efficiently performs the minimum necessary DOM mutations (insertions, updates, deletions) using browser DOM APIs (like `document.createElement`, `appendChild`).
- **Portals Support:** `ReactDOM.createPortal(child, domNode)` allows rendering components into a DOM node outside the parent component's DOM hierarchy (critical for modals, tooltips, dialogs).

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Source: React Official Docs; Unit 1 Slides (UNIT-1 ReactJS.pptx, slide 8)

Parameter	Imperative Programming (Vanilla JS)	Declarative Programming (React)
Philosophy	Describes <u>HOW</u> to achieve a result step-by-step.	Describes <u>WHAT</u> the UI should look like for a given state.
DOM Handling	Direct manual DOM manipulation (getElementById, innerHTML).	Virtual DOM automatically manages mutations based on state.
State Sync	Error-prone; state and DOM easily desynchronize.	Predictable; UI is a pure function of state: $UI = f(\text{state})$.

Unit 1 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 1 § 3](#)Explain how JSX is compiled internally with Babel transformations.Practice Problem:

Model Exam Answer

Solution:**1. JSX Compilation Process**

Browsers cannot understand JSX directly because it is not standard ECMAScript. During the build process, a JavaScript compiler such as **Babel** transforms JSX into plain JavaScript function calls.

2. Classic Runtime: `React.createElement`

Each JSX tag compiles into a call to `React.createElement(type, props, ...children)`:

- **Type:** A string for HTML tags (`'div'`, `'h1'`) or a function/class reference for custom components (App).
- **Props:** An object containing all attributes and event handlers (or `NULL` if none).
- **Children:** Any nested JSX elements, text strings, or expressions passed as rest parameters.

3. Transformation Example:**Input JSX:**

```
const element = (  
  <div className="card">  
    <h1>Welcome</h1>  
    <p>Hello Aki</p>  
  </div>  
);
```

Babel Transpiled Output:

```
const element = React.createElement(  
  'div',  
  { className: 'card' },  
  React.createElement('h1', null, 'Welcome'),  
  React.createElement('p', null, 'Hello Aki')  
);
```

Unit 1 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 1 § 3](#)

Discuss JSX syntax rules with suitable examples.

Practice Problem:

Model Exam Answer

Solution:

Fundamental Rules of JSX Syntax

1. **Single Root Element:** Every JSX expression must return a single top-level parent element. Multiple sibling elements must be wrapped in a container tag (e.g. `<div>`) or a React Fragment (`<> ... </>`):

```
// Correct:
return (
  <>
    <h1>Header</h1>
    <p>Paragraph</p>
  </>
);
```

2. **Close All Tags:** All elements must be explicitly closed. Self-closing tags must end with `/>`:

```

<input type="text" />
<br />
```

3. **camelCase Property Naming:** Because JSX compiles to JavaScript objects, HTML attribute names that collide with JS reserved words use camelCase:

- `class` → `className`
- `for` → `htmlFor`
- `tabindex` → `tabIndex`
- `onclick` → `onClick`

4. **JavaScript Expressions in Curly Braces:** Any valid JavaScript variable, function call, or expression can be embedded inside `{ }`:

```
const user = 'Aki';
return <h2>Hello, {user.toUpperCase()}!</h2>;
```

Unit 1 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 1 § 5](#)

Write a functional component that accepts props for name and marks and displays grade.

Practice Problem:

Model Exam Answer

Solution:

1. Component Implementation

```
import React from 'react';

function StudentReport({ name, marks }) {
  // Calculate letter grade based on marks
  let grade = 'F';
  if (marks ≥ 90) grade = 'A+';
  else if (marks ≥ 80) grade = 'A';
  else if (marks ≥ 70) grade = 'B';
  else if (marks ≥ 60) grade = 'C';
  else if (marks ≥ 50) grade = 'D';

  const isPass = marks ≥ 50;

  return (
    <div style={{
      border: '1px solid #cbd5e1',
      borderRadius: '8px',
      padding: '16px',
      maxWidth: '300px',
      fontFamily: 'sans-serif'
    }}>
      <h3 style={{ margin: '0 0 8px 0', color: '#0f172a' }}>Student
Report</h3>
      <p><strong>Name:</strong> {name}</p>
      <p><strong>Marks:</strong> {marks} / 100</p>
      <p>
        <strong>Grade:</strong>{' '}
        <span style={{ color: isPass ? '#16a34a' : '#dc2626',
fontWeight: 'bold' }}>
          {grade} ({isPass ? 'Passed' : 'Failed'})
        </span>
      </p>
    </div>
  );
}
```

Feature	Normal Props (Attributes)	props.children
Syntax	Passed as named attributes: <Component title="Hello" />	Passed as nested elements between open/closing tags: <Card> Body </Card>
Access	Accessed via property name: props.title	Accessed via special reserved property: props.children
Content Flexibility	Typically strings, numbers, booleans, objects, or callbacks.	Accepts arbitrary nested JSX structures, text, or multiple components (component composition).
Primary Use Case	Configuration and parameterized data.	Layout wrappers, modals, containers, and card shells.

Unit 1 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 1 § 5](#)

Write a Parent and Child component where parent passes a message through props.

Practice Problem:

Model Exam Answer

Solution:

1. Implementation Code

```
import React from 'react';

// Child Component: receives message via props and displays it
function MessageBanner({ message, sender }) {
  return (
    <div style={{
      padding: '12px',
      backgroundColor: '#f0fdf4',
      border: '1px solid #86efac',
      borderRadius: '6px',
      margin: '8px 0'
    }}>
      <h4 style={{ margin: '0 0 4px 0', color: '#166534' }}>From:
      {sender}</h4>
      <p style={{ margin: 0, color: '#14532d' }}>{message}</p>
    </div>
  );
}

// Parent Component: defines state/data and passes to child
function Dashboard() {
  const welcomeMsg = 'Welcome to Semester V CIE-1 Preparation Portal!';
  const faculty = 'ISE Department';

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h2>Parent Dashboard</h2>
      { /* Passing data down to Child via props */ }
      <MessageBanner message={welcomeMsg} sender={faculty} />
    </div>
  );
}

export default Dashboard;
```

Unit 1 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 1 § 6](#)

Explain how state flows inside a React application with an example.

Practice Problem:

Model Exam Answer

Solution:

1. One-Way Data Flow (Unidirectional Flow)

In React, state follows a strict **top-down (unidirectional) data flow**:

- **Single Source of Truth:** State is owned and maintained by a specific component.
- **Downward Flow:** State is passed down to child components exclusively via **props**.
Child components cannot directly alter parent state.
- **Upward Communication via Callbacks:** If a child needs to trigger a state change, the parent passes down a callback function via props. The child invokes the callback with updated values, allowing the parent to update its own state, triggering a clean re-render cycle.

2. Illustrative Example

```
import React, { useState } from 'react';

// Child component receives value and update handler
function ToggleButton({ isOn, onToggle }) {
  return (
    <button onClick={onToggle}>
      Status: {isOn ? 'ACTIVE' : 'INACTIVE'}
    </button>
  );
}

// Parent holds state and passes it down
function App() {
  const [isActive, setIsActive] = useState(false);

  const handleToggle = () => {
    setIsActive(prev => !prev); // Parent updates state
  };

  return (
    <div>
      <h3>Parent State: {isActive ? 'Running' : 'Stopped'}</h3>
    </div>
  );
}
```

Unit 1 5 Marks High Priority

SEE JAN 2026 CIE-1 2025

Theory Link: Unit 1 § 2

What is the Virtual DOM in React? Explain how the Reconciliation Diffing algorithm optimizes UI performance.

Practice Problem:

Model Exam Answer

Solution:

1. Definition of Virtual DOM

The **Virtual DOM (VDOM)** is a lightweight, in-memory JavaScript object representation of the real browser DOM. Directly manipulating the real DOM is slow and expensive because each mutation triggers layout recalculation, style recalculation, and page re-rendering (reflow and repaint).

2. The Reconciliation Diffing Algorithm ($O(n)$)

When component state changes, React creates a new Virtual DOM snapshot and compares it with the previous snapshot. React achieves an $O(n)$ heuristic comparison based on two key assumptions:

1. **Different Element Types Produce Different Trees:** If a root element changes type (e.g. from `<div>` to `<span>`), React unmounts the entire old tree and mounts a new tree from scratch without attempting to diff children.
2. **Keys Identify Stable Children:** When list items change, React uses unique **key** props to match children across renders, determining whether an item was inserted, deleted, or reordered without tearing down all nodes.

3. Batching & Patching

React calculates the minimal set of changes (the "diff") and applies them in a single batched update to the real DOM, ensuring 60 FPS UI responsiveness.

Source: React Official Docs (Reconciliation); Unit 1 Slides (UNIT-1 ReactJS.pptx, slides 14-17)

Lifecycle Phase	Class Component Method	Functional Component Hook Equivalent	Typical Purpose
Mounting	<code>componentDidMount()</code>	<code>useEffect(() ⇒ { ... }, [])</code> (empty dependency array)	API network calls, setting up WebSockets, listeners, DOM measurements
Updating	<code>componentDidUpdate(prevProps, prevState)</code>	<code>useEffect(() ⇒ { ... }, [dep1, dep2])</code>	Responding prop/state updates (e.g. re-fetching data on user change).
Unmounting	<code>componentWillUnmount()</code>	<code>useEffect(() ⇒ { return () ⇒ { cleanup(); } }, [])</code>	Clearing interval timers, canceling pending network requests, removing event listeners.

Unit 1 1 Marks Medium Priority

SEE JAN 2026 (Part A Q1)

Theory Link: Unit 1 § 4

React favors Composition over Inheritance. This means that instead of extending classes, UI parts are built by: (a) Combining reusable components, (b) Using imperative DOM updates, (c) Directly manipulating real DOM, (d) Writing new button elements.

Practice Problem:

Model Exam Answer

Solution:

Correct Answer: (a) Combining reusable components

Justification: React recommends using composition (nesting components and passing components as props/children) to reuse code and behavior between components rather than class inheritance hierarchies.

Source: React Official Docs (Composition vs Inheritance); JAN 2026 Q1

Unit 1 1 Marks Medium Priority

SEE JAN 2026 (Part A Q2)

Theory Link: Unit 1 § 3

The process of converting JSX code into calls to `React.createElement()` is known as the: (a) Diffing Algorithm, (b) Change Detection, (c) JSX Transform, (d) Virtual DOM Sync.

Practice Problem:

Model Exam Answer

Solution:

Correct Answer: (c) JSX Transform

Justification: Compilers like Babel perform the JSX Transform to convert JSX syntax into standard JavaScript function invocations.

Source: React Official Docs (Introducing JSX); JAN 2026 Q2

Unit 1 1 Marks Medium Priority

SEE JAN 2026 (Part A Q3)

Theory Link: Unit 1 § 7

Which lifecycle method is used to call APIs? (a) constructor, (b) render, (c) componentDidMount, (d) componentWillUnmount.

Practice Problem:

Model Exam Answer

Solution:

Correct Answer: (c) componentDidMount

Justification: Side-effects such as remote data fetching must occur after the component has mounted to the DOM so state updates can safely trigger re-renders.

Source: React Official Docs (State and Lifecycle); JAN 2026 Q3

Unit 1 1 Marks Medium Priority

SEE JAN 2026 (Part A Q4)

Theory Link: Unit 1 § 7

getSnapshotBeforeUpdate is used in the: (a) Mounting phase, (b) Updating phase, (c) Unmounting, (d) Error handling.

Practice Problem:

Model Exam Answer

Solution:

Correct Answer: (b) Updating phase

Justification: getSnapshotBeforeUpdate is invoked right before the most recently rendered output is committed to the DOM in the updating phase (e.g. to capture scroll position).

Source: React Official Docs (Component Specs); JAN 2026 Q4

Unit 2 Examination Questions & Answers

UNIT 2

Verified model solutions for Unit 2 past exam questions.

Unit 2 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 2 § 4](#)Describe event bubbling in React with an example.Practice Problem:

Model Exam Answer

Solution:**1. Event Bubbling in React & SyntheticEvent**

In standard DOM event propagation, an event triggered on a deeply nested element bubbles up through its ancestors in the DOM tree. React implements a cross-browser wrapper called **SyntheticEvent** and attaches top-level event listeners to the root container.

- When an event fires on a child (e.g. button click), the event bubbles upwards through React's Virtual DOM hierarchy.
- Parent handlers on the path are invoked unless explicitly stopped via `e.stopPropagation()`.

2. Code Example

```
import React from 'react';

function BubblingDemo() {
  const handleParentClick = () => {
    alert('Parent DIV clicked (Bubbled up)!');
  };

  const handleChildClick = (e) => {
    alert('Child BUTTON clicked!');
    // Uncomment line below to prevent event from bubbling to parent:
    // e.stopPropagation();
  };

  return (
    <div onClick={handleParentClick} style={{ padding: '20px',
background: '#e2e8f0' }}>
      <p>Parent Container</p>
      <button onClick={handleChildClick}>Click Me</button>
    </div>
  );
}
```

Clicking the button triggers "Child BUTTON clicked!" first, followed by "Parent DIV clicked

Unit 2 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 2 § 5](#)

Build a functional component where users can type into an input box, and the app displays the current character count below it.

Practice Problem:

Model Exam Answer

Solution:

1. Component Implementation

```
import React, { useState } from 'react';

function CharacterCounter() {
  const [text, setText] = useState('');
  const maxLimit = 100;

  const handleChange = (e) => {
    setText(e.target.value);
  };

  const charCount = text.length;
  const isOverLimit = charCount > maxLimit;

  return (
    <div style={{ maxWidth: '400px', padding: '16px', fontFamily:
'sans-serif' }}>
      <label htmlFor="user-input" style={{ fontWeight: 'bold',
display: 'block', marginBottom: '6px' }}>
        Enter your text:
      </label>
      <input
        id="user-input"
        type="text"
        value={text}
        onChange={handleChange}
        placeholder="Type something..."
        style={{
          width: '100%',
          padding: '8px 12px',
          borderRadius: '6px',
          border: `1px solid ${isOverLimit ? '#ef4444' : '#cbd5e1'}`,
          boxSizing: 'border-box'
        }}
      />
    </div>
  );
}
```

Unit 2 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 2 § 5](#)

Discuss handling inputs like checkbox, and textarea with examples.

Practice Problem:

Model Exam Answer

Solution:

1. Handling Diverse Form Inputs in React

In React, all form inputs become **controlled components** when their display value is tied to React state and updated via an onChange handler.

- **Textarea:** In HTML, <textarea> takes text between tags. In React, it uses a value prop identically to single-line text inputs.
- **Checkbox:** Controlled using the checked boolean attribute (not value) and checked state is accessed via e.target.checked.

2. Multi-Input Form Code Example

```
import React, { useState } from 'react';

function FeedbackForm() {
  const [formData, setFormData] = useState({
    feedback: '',
    subscribe: false
  });

  const handleChange = (e) => {
    const { name, type, checked, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  return (
    <form onSubmit={e => e.preventDefault()}>
      <label>Your Comments:</label>
      <textarea
        name="feedback"
        value={formData.feedback}
        onChange={handleChange}
        rows="3"
      />
    </form>
  );
}
```

Unit 2 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 2 § 6](#)*Why is lifting state up required? Provide a real-time example.**Practice Problem:*

Model Exam Answer

*Solution:**1. Why Lifting State Up is Required*

Often, two or more sibling components need to share or react to the same changing data. In React's unidirectional data flow, sibling components cannot pass state directly sideways to each other. To keep data synchronized, developers *lift state up* to their closest common ancestor component.

2. Real-Time Example: Temperature Converter (Celsius & Fahrenheit)

```
import React, { useState } from 'react';

// Child Component: Reusable Temperature Input
function TemperatureInput({ scale, temperature, onTemperatureChange }) {
  return (
    <fieldset style={{ margin: '8px 0' }}>
      <legend>Enter temperature in {scale === 'c' ? 'Celsius' : 'Fahrenheit'}:</legend>
      <input
        value={temperature}
        onChange={e ⇒ onTemperatureChange(e.target.value)}
        type="number"
      />
    </fieldset>
  );
}

// Common Parent: Holds shared state and performs synchronization
function Calculator() {
  const [temperature, setTemperature] = useState('');
  const [scale, setScale] = useState('c');

  const handleCelsiusChange = (val) ⇒ {
    setScale('c');
    setTemperature(val);
  };
}
```

Unit 2 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 2 § 2](#)Explain useState with an example of updating state objects and arrays.Practice Problem:

Model Exam Answer

Solution:**1. The useState Hook & Immutability**

The useState hook lets functional components declare state variables. In React, **state must be treated as immutable**. Mutating an object or array in place (e.g. `arr.push()` or `obj.name = 'X'`) does not change the object's reference memory address, causing React to miss the update and fail to re-render.

2. Correct Immutable Update Patterns

```
import React, { useState } from 'react';

function ProfileEditor() {
  // 1. Object State
  const [user, setUser] = useState({ name: 'Aki', sem: 5, college: 'MSRIT' });

  // 2. Array State
  const [courses, setCourses] = useState(['CN', 'ReactJS', 'AI']);

  // Updating Object: Use Object Spread (...)
  const updateCollege = () => {
    setUser(prev => ({
      ...prev,           // Copy existing properties
      college: 'Ramaiah Institute of Technology'
    }));
  };

  // Updating Array: Use Array Spread (...) or filter/map
  const addCourse = () => {
    setCourses(prev => [...prev, 'SE']); // Append without mutating original
  };

  const removeCourse = (courseToRemove) => {
    setCourses(prev => prev.filter(c => c !== courseToRemove)); // Return new array
  };
}
```

Unit 2 4 Marks High Priority

SEE JAN 2026

[Theory Link: Unit 2 § 2](#)

Create a component that shows current time and updates every second using `useEffect`.

Practice Problem:

Model Exam Answer

Solution:

1. Implementation Code: Live Clock with Interval Cleanup

```
import React, { useState, useEffect } from 'react';

function DigitalClock() {
  const [time, setTime] = useState(new Date().toLocaleTimeString());

  useEffect(() => {
    // Setup: start 1-second interval timer when component mounts
    const timerId = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);

    // Cleanup: clear timer when component unmounts to prevent memory leak
    return () => {
      clearInterval(timerId);
    };
  }, []); // Empty dependency array [] = run effect once on mount

  return (
    <div style={{
      display: 'inline-block',
      padding: '16px 24px',
      backgroundColor: '#0f172a',
      color: '#38bdf8',
      borderRadius: '8px',
      fontFamily: 'monospace',
      fontSize: '1.5rem',
      boxShadow: '0 4px 6px rgba(0,0,0,0.1)'
    }}>
      <div style={{ fontSize: '0.8rem', color: '#94a3b8', textTransform: 'uppercase' }}>
        Live Time
      </div>
      <div>{time}</div>
    </div>
  );
}
```

Unit 2 4 Marks High Priority

CIE-1 2025

Theory Link: Unit 2 § 1

State the two Rules of Hooks in React and explain why these rules must be strictly followed.

Practice Problem:

Model Exam Answer

Solution:

1. The Two Golden Rules of Hooks

- 1. Only Call Hooks at the Top Level:** Do not call hooks inside loops, conditions, or nested functions. Always use Hooks at the top level of your React function component before any early returns.
- 2. Only Call Hooks from React Functions:** Only call Hooks from React function components or from custom Hooks. Do not call Hooks from regular JavaScript functions.

2. Rationale: Hook Call Order Preservation

- React does not rely on variable names to identify state. Instead, React maintains an **internal linked-list / array of hook state cells** for each component instance.
- React relies on the exact **order** in which Hooks are called on every single render to match each Hook call with its corresponding internal state slot.
- If a Hook is placed inside an `if` condition that evaluates to false on a subsequent render, the Hook call order is shifted, destroying state synchronization and causing runtime crashes.

Source: React Official Docs (Rules of Hooks); Unit 2 Slides (Unit-2.pdf, page 5)

Unit 2 5 Marks Medium Priority

CIE-1 2025

[Theory Link: Unit 2 § 3](#)**What is Prop Drilling? How does the useContext hook solve this problem?****Practice Problem:**

Model Exam Answer

Solution:**1. Prop Drilling Problem**

Prop drilling occurs when data must be passed from a high-level parent component to a deeply nested child component through multiple intermediate components that do not actually need the data themselves, acting solely as pass-through conduits.

2. Solution: React Context & useContext

Context provides a way to pass data through the component tree without having to pass props down manually at every level.

```
import React, { createContext, useContext, useState } from 'react';

// 1. Create Context
const ThemeContext = createContext('light');

// 2. Provider Component
function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <PageLayout /> {/* Intermediate component does not pass theme */}
    </ThemeContext.Provider>
  );
}

// 3. Deep Consumer Component
function ThemedButton() {
  // Access theme directly without prop drilling!
  const theme = useContext(ThemeContext);
  return <button className={`btn-${theme}`}>Current Theme: {theme}</button>;
}
```

Source: React Official Docs (Passing Data Deeply with Context); Unit 2 Slides (Unit-2.pdf, pages 19-21)

Parameter	Controlled Components (Recommended)	Uncontrolled Components
State Source	Managed directly by React state (useState).	Managed by the DOM itself (internal DOM state).
Value Access	Values accessed via React state variables.	Values accessed via useRef() DOM reference.
Form Validation	Instant real-time validation on every keystroke.	Validation typically performed only at form submission.
Example Syntax	<pre><input value={val} onChange={e ⇒ setVal(e.target.value)} /></pre>	<pre><input type="text" ref= {inputRef} defaultValue="Initial" /></pre>

Unit 2 1 Marks Medium Priority

SEE JAN 2026 (Part A Q5)

[Theory Link: Unit 2 § 4](#)

Inline event handlers are: (a) Recommended for reusable logic, (b) Good for small actions, (c) Not allowed, (d) Replace addEventListener.

Practice Problem:

Model Exam Answer

Solution:

Correct Answer: (b) Good for small actions

Justification: Arrow function inline handlers like `onClick={() ⇒ setCount(c + 1)}` are concise and ideal for quick single-line state mutations.

Source: React Official Docs (Responding to Events); JAN 2026 Q5

Unit 2 1 Marks Medium Priority

SEE JAN 2026 (Part A Q6)

[Theory Link: Unit 2 § 5](#)Which of the following is a controlled component? (a) , (b){x} , (c) No file chosen, (d) Practice Problem:

Model Exam Answer

Solution:**Correct Answer:** (b) `<input value={x} onChange={f} />`**Justification:** In a controlled component, the input value is bound to a state variable x and modified via an event handler f.**Source:** React Official Docs (Controlled Components); JAN 2026 Q6

Out of CIE-1 Scope Questions (10 Questions)

ARCHIVED

The following questions were identified in past papers but fall outside the official CIE-1 syllabus. They are preserved here to ensure zero questions are dropped.

Paper / Origin	Question Summary	Exclusion Rationale
SEE JAN 2026 Part A Q7	useParams returns: (a) URL parameters, (b) Query string, (c) Location object, (d) Component props.	<u>useParams is part of React Router (Unit 4 / Unit 3 in older scheme), out of CIE-1 scope.</u>
SEE JAN 2026 Part A Q8	Link component prevents: (a) JSX errors, (b) Page reload, (c) DOM mutation, (d) API calls.	<u>React Router is out of CIE-1 scope.</u>

Paper / Origin	Question Summary	Exclusion Rationale
SEE JAN 2026 Part A Q9	Environment variables in React must start with: (a) VAR_, (b) ENV_, (c) REACT_APP_, (d) APP_ENV_.	<u>Production build and deployment configuration belongs to Unit 5.</u>
SEE JAN 2026 Part A Q10	Which step generates optimized JavaScript/CSS files? (a) Development mode, (b) Production build, (c) Testing mode, (d) Node server.	<u>Webpack / Vite production build optimization is part of Unit 5.</u>
SEE JAN 2026 Q7a	Discuss performance optimization using React DevTools Profiler.	<u>React DevTools Profiler belongs to Unit 4.</u>
SEE JAN 2026 Q7b	Create a multi-page app with Home, Services, Contact pages using Routes.	<u>Multi-page client-side routing with react-router-dom belongs to Unit 4.</u>
SEE JAN 2026 Q8a	Create a navigation bar that highlights the active page using NavLink.	<u>NavLink active styles belong to React Router in Unit 4.</u>
SEE JAN 2026 Q8b	Explain how dynamic routing works using :id parameters.	<u>Dynamic route parameters belong to React Router in Unit 4.</u>
SEE JAN 2026 Q9a	Explain the purpose of .env file and how environment variables are used in React.	<u>.env configuration belongs to Unit 5.</u>
SEE JAN 2026 Q9b	Create a card layout where top-level styles are global but component styles use CSS Modules.	<u>CSS Modules and styling architecture belongs to Unit 5.</u>